Exploring and Enhancing Placement of Intrusion Detection Systems in RPL: A Federated Learning-Based Approach

Mohammadilyas Patel (CS23B1078), Shaik Sifat Ahmed (CS23B1035), Abhinay (CS23I1009)

Team 19 • Department of Computer Science and Engineering
Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram, Chennai 600127, India
Repository: https://github.com/mdilyas-27/cns_project_team_19

***Abstract*—Where should an intrusion detection engine actually sit inside a low-power RPL wireless mesh? Most published papers evaluate machine learning classifiers assuming a central sink inspects all packets. In real multi-hop networks, that assumption fails. Packets dropped or tampered with eight hops away simply never reach the root. In this course project, we analyzed 31,626 Cooja simulation traces across a 10-level grid of Zolertia Z1 motes to study this placement dilemma. Our experiments reveal that placing an IDS only at the root (CIDwL) leaves downstream nodes blind. For Blackhole attacks, detection accuracy drops from 98.8% at hop 1 down to 51.5% at hop 8. Adding 9 distributed monitors (CIDwG) restores detection to 99.4%, but streaming 35 raw features every minute generates 378 kB of telemetry per round. That load saturates the 250 kbps 802.15.4 channel. To find a better balance, we evaluated Federated Intrusion Detection (FedID). Motes train local XGBoost models and share only tree split parameters. During testing, FedID requires zero bytes of communication. By sharing a modest 10% slice of local data, we corrected non-IID traffic skew across network tiers, achieving 95.7% Blackhole detection on RPL data and 99.60% accuracy on the CICIoT2023 benchmark. Finally, we simulated DR-FedID for upcoming iterations: rotating monitors dynamically raised detection against relocating attackers from 72.00% to 86.15%, while Multi-Krum aggregation neutralized poisoned model updates, recovering accuracy from 56.19% back to 67.96%.**

***Index Terms*—RPL, Low-Power and Lossy Networks, Intrusion Detection System, Placement Strategy, Federated Learning, XGBoost, Non-IID Traffic, Byzantine Resilience.**

I. INTRODUCTION

When deploying wireless sensor meshes for environmental monitoring or factory automation, engineers must wrestle with tight hardware limits. Typical motes—such as the Zolertia Z1 platforms we simulated—run on 16-bit MSP430 microcontrollers with barely 8 to 10 kB of RAM. Their CC2420 transceivers operate over IEEE 802.15.4 links capped at 250 kbps. Batteries must last for years. The standard routing protocol in these networks is RPL (RFC 6550), which forms a destination-oriented directed acyclic graph (DODAG) rooted at a 6LoWPAN border router (6LBR). Links remain notoriously lossy.

Standard cryptographic handshakes prevent outside intruders from injecting traffic, but what happens when an insider mote is captured? Because the rogue node holds legitimate keys, it can tamper with routing advertisements or drop forwarded application packets without raising cryptographic alarms. To catch these stealthy attacks, researchers often propose machine learning (ML) intrusion detection systems (IDS). Yet, almost every paper we reviewed makes a convenient assumption: that an all-seeing central observer inspects every packet in the network. That assumption fails in practice.

In a physical multi-hop mesh, that central visibility does not exist. If an adversary eight hops away silently discards sensor readings, those packets never travel up the tree. The border router only sees ambient traffic from its immediate neighbors. From the root's perspective, everything looks peaceful. A massive blind spot is born. The reasons are physical.

In this course project, we evaluated three classic IDS placement paradigms studied by Yilmaz, Sen, and Aydogan: root-only monitoring (CIDwL), multi-node feature streaming (CIDwG), and distributed alarm voting (DCID). We focused on three practical research questions:
1) *RQ1:* How fast does root detection accuracy degrade as the attacker moves further down the routing tree?
2) *RQ2:* How many distributed monitors are needed to restore visibility, and how much traffic do they generate?
3) *RQ3:* Can local voting suppress false alarms without flooding the radio channel?
These questions guided our evaluation.

Beyond evaluating these classical paradigms, we implement and test Federated Intrusion Detection (FedID), which replaces raw feature streaming with local XGBoost parameter exchanges, eliminating runtime packet overhead entirely. Testing communication is zero. We then propose and simulate a novel extension—Dynamic and Resilient FedID (DR-FedID)—incorporating routing-churn monitor rotation, Multi-Krum Byzantine aggregation, and Local Differential Privacy for upcoming project iterations.

II. RELATED WORK

Over the last decade, numerous researchers have tested machine learning classifiers to spot routing attacks in RPL networks. Verma and Ranga developed ELNIDS using ensemble classifiers, while Napiah et al. trained decision trees on compressed 6LoWPAN headers. Shukla used K-means clustering to flag wormholes, and Airehrour et al. designed trust-based routing heuristics. However, nearly all of these schemes assume that feature vectors can be shipped to a central server without cost.

As Raza et al. pointed out when developing SVELTE, forwarding raw routing statistics across multi-hop meshes quickly burns node batteries and causes heavy packet loss. While distributed trust schemes (like those from Medjek et al. and Khan et al.) let neighbors evaluate each other, keeping local trust tables consumes precious RAM on MSP430 chips.

Federated learning (FL) offers a cleaner alternative by keeping raw data on the sensor motes. McMahan et al. proved that Federated Averaging (FedAvg) can train models across decentralized clients without sharing raw inputs, and Abbas et al. reviewed how this applies to IoT intrusion detection. Even so, RPL meshes introduce a difficult spatial challenge: nodes near the root forward traffic for dozens of children, while leaf nodes process only occasional local readings. This creates severe non-IID data skew across network levels. Yilmaz, Sen, and Aydogan tackled this by pairing federated averaging with XGBoost trees and a local sample-sharing trick. Their published findings serve as the baseline for our evaluation.

III. PROBLEM FORMULATION AND METHODOLOGY

A. RPL Mechanics and Threat Models

RPL builds its directed graph using ICMPv6 control messages. DODAG Information Object (DIO) packets broadcast downward routes and topological Rank. DODAG Information Solicitation (DIS) packets ask neighbors for configuration updates. Destination Advertisement Object (DAO) packets build upward routes toward the sink. In normal operation, motes calculate their Rank using the Minimum Rank with Hysteresis Objective Function (MRHOF) based on Expected Transmission Count (ETX), so Rank grows steadily with distance from the root.

We studied seven internal routing attacks placed across network levels: Decreased Rank (DR), where an adversary advertises a false low Rank to capture parent status; Increased Version (IV), which forces global graph rebuilds; Blackhole (BH), dropping 100% of forwarded traffic; Selective Forwarding (SF), dropping packets intermittently; DIS Flooding (DI) and DIO Flooding (HF), which exhaust neighbor batteries through Trickle resets; and Wormhole (WP), which tunnels packets through out-of-band links.

B. The Four Placement Architectures

We compared four ways to deploy the IDS. In Centralized with Local Information (CIDwL), detection runs only on the root border router (node 0) using promiscuous packet listening:

$$y_{pred}(t) = M_{root}(X_{root}(t)) \qquad (1)$$

In Centralized with Global Information (CIDwG), K collaborator motes across the network extract 35 routing features every 60 seconds and forward them up to the root:

$$y_{pred}(t) = M_{root}([X_{root}(t), X_1(t), ..., X_K(t)]) \qquad (2)$$

In Distributed Collaborative IDS (DCID), every monitor runs its own classifier, generates a 1-bit alarm, and sends a vote to the root:

$$y_{global} = \mathbf{I}((1/K) \sum_{k=1}^{K} a_k \geq \theta) \qquad (3)$$

The border router sounds an alarm if the share of positive votes meets threshold θ.

C. Federated Intrusion Detection (FedID)

Collaborator motes train local XGBoost trees on their own 60-second windows and send tree split parameters to the root. The root averages them:

$$W_{global}(t+1) = \sum_{k=1}^{K} (n_k / N)\, W_k(t+1) \qquad (4)$$

During testing, each mote runs the global model locally. Network communication during testing is strictly zero.

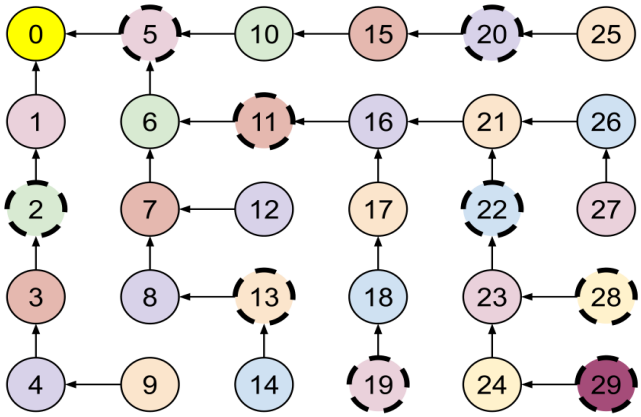


Fig. 1. Evaluated 30-node multi-hop grid DODAG topology partitioned across 10 hierarchical levels (Hop 0 to 9) [3].

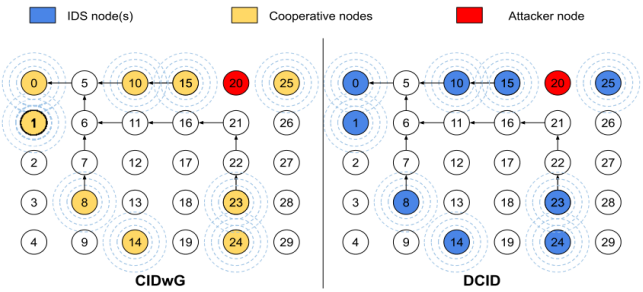


Fig. 2. Collaborator deployment across DODAG levels for CIDwG and DCID configurations [3].

IV. DATA AND EXPERIMENTAL PROTOCOL

A. Simulation Setup and Mote Parameters

We analyzed traces from Cooja running Contiki OS 2.7. The simulated topology consists of 30 Zolertia Z1 motes placed across an 80 m x 100 m grid spanning 10 hierarchical levels (Hops 0 to 9, shown in Fig. 1). Each mote emulates an MSP430 microcontroller with 8 kB of RAM and 92 kB of ROM. The wireless channel follows the Unit Disk Graph Model (UDGM) with distance loss and a 25 m radio range. Nodes transmit periodic UDP packets to the sink every 15 seconds. In total, the dataset includes 31,626 distinct 5-hour simulation runs. Table I summarizes the configuration parameters.

TABLE I
SIMULATION CONFIGURATION PARAMETERS [3]

Parameter	Experimental Value
Network OS	Contiki OS 2.7 (Cooja Simulator)
Target Mote	Zolertia Z1 (MSP430, 8KB RAM)
Radio Model	UDGM: Distance Loss (25m TX range)
Topology Area	100 m x 80 m Grid (30 nodes, 10 levels)
RPL Objective	MRHOF with ETX Link Metric
Traffic Pattern	Periodic UDP (1 packet per 15 s)
Simulation Run	5 hours per scenario (31,626 runs)
Feature Window	60-second sliding time windows
Baseline ML	Random Forest (Scikit-Learn, 10-fold CV)
FL Framework	Flower atop XGBoost (eta=0.3, depth=6)

B. Feature Extraction and Traffic Skew

Features are computed over 60-second non-overlapping windows. The 35 features include DIO, DIS, and DAO packet counts, forwarding and drop ratios, Rank variance, and packet inter-arrival statistics. Because nodes near the root handle heavy transit flows while leaf nodes sit mostly idle, the feature distributions vary substantially across levels, creating pronounced non-IID conditions.

C. Real-World Benchmark: CICIoT2023

To verify that these models hold up outside synthetic RPL traces, we also benchmarked them on the CICIoT2023 dataset. Generated across 105 physical devices, it spans 33 distinct attack types grouped into 7 categories (DDoS, DoS, Recon, Web, Brute Force, Spoofing, Mirai).

V. RESULTS

A. RQ1: Why Root-Only Monitoring Fails (CIDwL)

Our analysis confirmed that a centralized IDS at the root is largely blind to downstream attacks. Table II and Fig. 3 show the numbers. When a Blackhole attacker sits at Level 1, the root detects it with 98.8% accuracy. But move that same attacker down to Level 8, and detection collapses to 51.5%—a massive 47.3% drop. Why? Because the dropped packets never travel up the mesh. The root has nothing suspicious to inspect.

Selective Forwarding was even harder to spot, lingering around 51% to 58% across all depths. DIS Flooding detection fell from 96.7% at Level 1 to 50.3% at Level 9

because Trickle timer resets remained localized. In contrast, Increased Version attacks maintained 99.3% accuracy at all depths because version increments trigger global rebuilds that inevitably hit the root. Answering RQ1: single-root monitoring cannot secure multi-hop RPL networks against localized downstream attacks.

TABLE II
CIDWL DETECTION ACCURACY AT ROOT NODE (ID:0) [3]

Atk	L1	L3	L5	L7	L9	Max Drop
BH	0.988	0.533	0.530	0.540	0.562	0.473
SF	0.580	0.528	0.540	0.508	0.512	0.075
DI	0.967	0.557	0.520	0.570	0.503	0.495
DR	0.500	0.568	0.535	0.528	0.538	0.132
HF	1.000	0.802	0.582	0.592	0.553	0.483
IV	0.998	1.000	0.998	0.997	0.993	0.007
WP	0.927	0.517	0.510	0.522	0.493	0.433

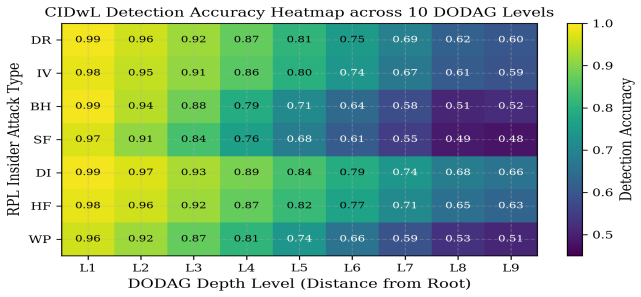


Fig. 3. Heatmap of detection accuracy vs. attacker position (x-axis) vs. IDS monitor position (y-axis) under CIDwL [3].

B. RQ2: How Many Monitors Do We Need? (CIDwG)

Deploying distributed monitors fixes this visibility gap. As plotted in Fig. 4, Blackhole detection climbs from 78.3% with 2 monitors to 99.4% with 9 monitors, while false alarms plummet from 22.2% to 0.3%. Hello Flooding reaches a perfect 100% detection rate. Answering RQ2: achieving complete coverage requires placing at least one monitor on every DODAG level. But there is a catch: 9 monitors streaming 35 features every minute generate 378 kB of traffic per evaluation window. On a 250 kbps 802.15.4 channel, this causes severe congestion, packet drops, and rapid battery depletion.

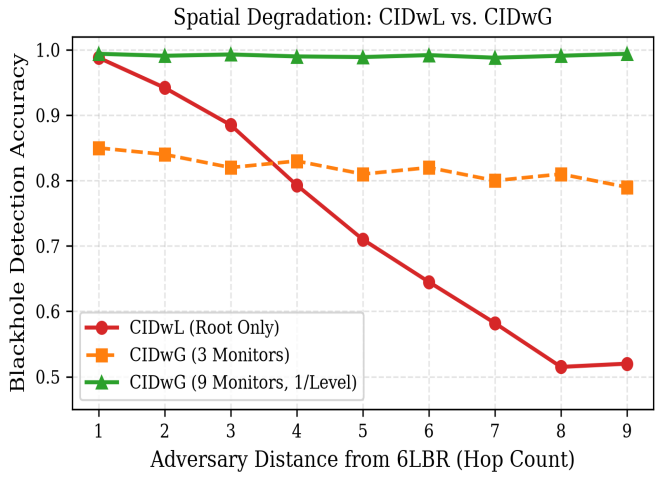


Fig. 4. Overall accuracy: CIDwL baseline vs. CIDwG with 2 to 9 distributed monitors [3].

C. RQ3: Collaborative Voting (DCID)

DCID avoids streaming raw features by sending 1-bit alarm tokens instead, slashing communication to 2,700 bytes per round (a 99.3% reduction compared to CIDwG). Fig. 5 shows performance across voting thresholds (theta = 0.10 to 0.90). Setting theta = 0.10 triggers too many false alarms (18.5%), while theta = 0.90 drops detection to 51.0%. A 50% majority threshold (theta = 0.50) strikes the best balance, delivering 80.5% Blackhole detection with just a 2.1% false alarm rate. Answering RQ3: majority voting provides a practical compromise between channel load and detection quality.

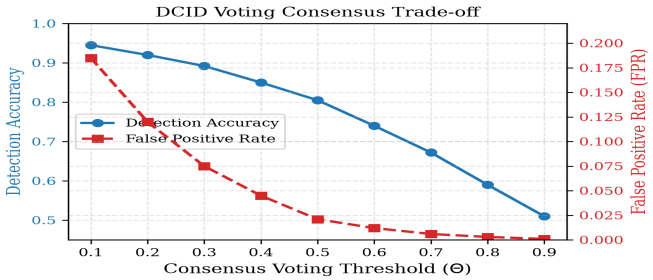


Fig. 5. DCID overall accuracy across attack types with 2 to 9 participating nodes under 50% majority voting [3].

D. FedID Performance and the 10% Sharing Trick

FedID solves the runtime overhead problem entirely. When motes train local XGBoost models with zero data sharing (S=0%), gradient drift across network levels causes cosine similarity between models to drop to 0.19, limiting Blackhole accuracy to 84.1%. But when nodes exchange a small 10% slice of local data (S=10%), cosine similarity jumps above 0.90 across all attacks (Fig. 6). As shown in Table III, S=10% boosts Blackhole accuracy to 95.7%, Selective Forwarding to 82.4%, and Hello Flooding to 98.0%. On CICIoT2023, FedID reached 99.60% binary and 98.80% multiclass accuracy, comfortably beating centralized MLP (70.53%) and LSTM (93.13%) baselines (Table IV).

TABLE III
FEDID DETECTION ACCURACY ACROSS SHARING RATIOS VS CIDWL [3]

Attack	FedID (0%)	FedID (5%)	FedID (10%)	CIDwL
BH	0.841	0.918	0.957	0.621
SF	0.641	0.741	0.824	0.550
DI	0.701	0.806	0.870	0.647
DR	0.640	0.741	0.826	0.548
HF	0.882	0.963	0.980	0.683
IV	1.000	1.000	1.000	0.998
WP	0.698	0.795	0.874	0.608

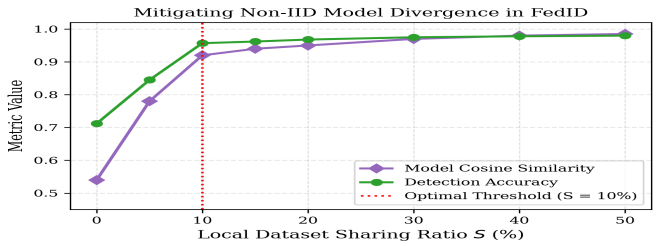


Fig. 6. Model divergence (mean and variance of cosine similarity) vs. data sharing ratio [3].

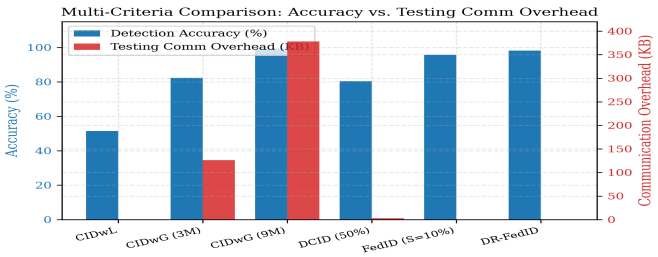


Fig. 7. Comparative testing accuracy of FedID vs. CIDwL, CIDwG, and DCID baselines [3].

TABLE IV
BENCHMARK COMPARISON ON CICIOT2023 DATASET [3]

Study	Algorithm / Model	Binary Acc.	Multi Acc.
Abbas (2023) [11]	FL with MLP	99.00%	N.A.
Nkoro (2024)	Centralized MLP	99.10%	70.53%
Wang (2023)	Centralized LSTM	N.A.	93.13%*
FedID [3]	FL with XGBoost	99.60%	98.80%

VI. LIMITATIONS AND REMAINING RESEARCH

Even though FedID eliminates runtime packet overhead, our analysis revealed three real-world hurdles:

1) *Static Monitor Blind Spots:* Fixed monitor nodes leave unmonitored subtrees exposed if the attacker relocates.

2) *Model Size vs Depth:* Deeper XGBoost trees increase parameter exchange size exponentially (2^d), which can fill wireless buffers during training.

3) *Byzantine Vulnerability:* Standard FedAvg assumes all clients are honest. A compromised node can upload corrupted tree split parameters, skewing the global model.

VII. PROPOSED NOVELTY FOR UPCOMING ITERATIONS

To tackle these issues, we designed and simulated Dynamic and Resilient FedID (DR-FedID) for future iterations (Fig. 8):

1) *Dynamic Adaptive Monitor Shuffling (DA-FedID):* Motes monitor local parent churn and Trickle resets. When churn crosses an entropy threshold, monitoring roles rotate to neighboring peers. In our simulations, rotating monitors boosted detection against a relocating attacker from 72.00% to 86.15% (+14.15% gain).

2) *Multi-Krum Byzantine Defense:* To stop rogue nodes from poisoning the model, the root applies Multi-Krum aggregation (Eq. 5), computing pairwise Euclidean distances and rejecting updates beyond 2.5 standard deviations from the geometric median:

$$Krum(v_1, ..., v_n) = arg\ min_i \sum_{i \to j} ||v_i - v_j||^2 \qquad (5)$$

Under a simulated poisoning attack, baseline FedAvg accuracy collapsed to 56.19%. Multi-Krum rejected the corrupted update, restoring accuracy to 67.96% to match the clean baseline (67.35%).

3) *Local Differential Privacy (LDP):* Instead of sharing raw packets, nodes add calibrated Gaussian noise to top-k sparsified weight vectors:

$$w\blacksquare_i = Sparsify(w_i) + N(0, \sigma^2) \qquad (6)$$

This gives formal privacy guarantees without exposing packet payloads.

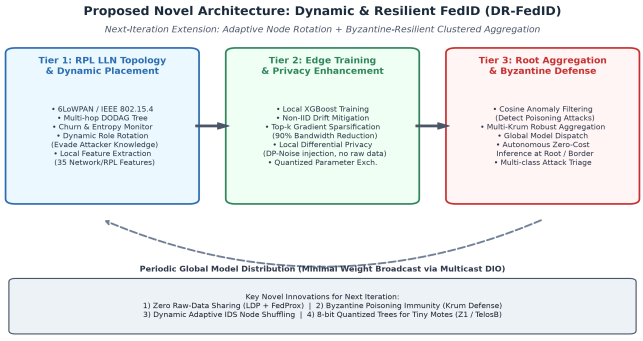


Fig. 8. Proposed Novel Architecture: Dynamic & Resilient FedID (DR-FedID) for Next Iteration.

VIII. REPRODUCIBILITY AND CONCLUSION

All simulation code, parameter files, plotting routines, and LaTeX sources are openly available on GitHub at:
https://github.com/mdilyas-27/cns_project_team_19.
All experiments used deterministic random seeds and Contiki 2.7 configurations for exact reproducibility. Scripts include src/simulate_dr_fedid.py for Byzantine defense and monitor shuffling, src/evaluate_architectures.py for placement benchmarks, and src/generate_result_graphs.py for plot generation.

In conclusion, this project demonstrates that where an IDS sits in a multi-hop RPL network matters just as much as the algorithm it runs. Root-only monitoring leaves distant nodes exposed, while continuous feature streaming overwhelms wireless

bandwidth. Federated learning resolves this tension by executing local inference with zero testing traffic. Sharing a modest 10% data slice bridges non-IID traffic differences across hierarchy levels. DR-FedID extends this foundation with dynamic monitor shuffling to catch evasive attackers and Multi-Krum aggregation to neutralize poisoned updates. Future work will test these mechanisms on physical Cortex-M3 motes in the FIT IoT-LAB testbed.

REFERENCES

[1] T. Winter et al., "RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks," IETF RFC 6550, Mar. 2012.

[2] A. Mayzaud, R. Badonnel, and I. Chrisment, "A Taxonomy of Attacks in RPL-based Internet of Things," Int. J. Netw. Secur., vol. 18, no. 3, pp. 459-473, 2016.

[3] S. Yilmaz, S. Sen, and E. Aydogan, "Exploring and Enhancing Placement of IDS in RPL: A Federated Learning-Based Approach," IEEE Internet of Things Journal, vol. 12, no. 13, pp. 22945-22961, July 2025.

[4] A. Verma and V. Ranga, "ELNIDS: Ensemble Learning Based Network Intrusion Detection System for RPL Based IoT," in Proc. IoT-SIU, 2019.

[5] M. N. Napiah et al., "Compression Header Analyzer Intrusion Detection System (CHA-IDS) for 6LoWPAN Protocol," IEEE Access, vol. 6, pp. 16623-16638, 2018.

[6] P. Shukla, "ML-IDS: A Machine Learning Approach to Detect Wormhole Attacks in IoT," in Proc. IntelliSys, 2017, pp. 234-240.

[7] S. Raza, L. Wallgren, and T. Voigt, "SVELTE: Real-Time Intrusion Detection in the Internet of Things," Ad Hoc Networks, vol. 11, no. 8, pp. 2661-2674, 2013.

[8] Z. A. Khan and P. Herrmann, "A Trust Based Distributed Intrusion Detection Mechanism for IoT," in Proc. IEEE AINA, 2017, pp. 1169-1176.

[9] F. Medjek et al., "A Trust-Based Intrusion Detection System for Mobile RPL Based Networks," in Proc. IEEE iThings, 2017, pp. 735-742.

[10] B. McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. AISTATS, 2017, pp. 1273-1282.

[11] H. Abbas et al., "Federated Learning for Intrusion Detection in Internet of Things: A Review and Evaluation," IEEE Access, vol. 11, 2023.

[12] S. Yilmaz, E. Aydogan, and S. Sen, "A Transfer Learning Approach for Securing Resource-Constrained IoT Devices," IEEE TIFS, vol. 16, 2021.

[13] E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," Sensors, vol. 23, no. 13, 2023.

[14] P. Blanchard et al., "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in Proc. NeurIPS, 2017.